

Week 4 Lab: Motor Control & H-Bridge

EE0827 Embedded System Applications

Overview

Build and test circuits for controlling DC motors safely. You will implement flyback protection, PWM speed control, and bidirectional operation using an H-bridge driver. These skills are essential for any project involving motors, solenoids, or other inductive loads.

Time allocation: 90 minutes

- MOSFET switching & flyback protection: 35 min
- PWM speed control: 20 min
- H-bridge operation: 25 min
- Documentation and cleanup: 10 min

Learning Objectives

By the end of this lab, you will be able to:

1. Switch a DC motor using a MOSFET with proper protection
2. Observe flyback voltage spikes and understand why protection is critical
3. Implement PWM speed control and measure motor response
4. Control motor direction using an H-bridge driver IC

Safety Gate

Safety Checklist (Complete Before Powering On)

- ☐ Bench supply current limit set to **500 mA** maximum
- ☐ Motor secured to breadboard or held firmly (prevents flying wires)
- ☐ MOSFET gate resistor in place (prevents oscillation)
- ☐ All connections verified against schematic
- ☐ Safety glasses recommended (motor can throw debris)

Motor Control Hazards

1. **Motors draw HIGH current at startup** - always set current limit first
2. **Flyback spikes stress and can destroy components** - always add protection in final designs
3. **Spinning motors are dangerous** - secure motor, keep fingers clear
4. **MOSFETs can get very hot** - if hot to touch, something is wrong
5. **Never connect motor directly to GPIO** - always use driver circuit

Required Parts and Tools

Hardware

- STM32 Nucleo-F401RE board
- Bench power supply (6-12V, current limiting)
- Small DC motor (6V hobby motor from kit)
- N-channel MOSFET (IRLZ44N or logic-level equivalent)
- Schottky diode (1N5819 or similar)
- Motor driver module (TB6612FNG or L298N)
- Resistors: 1k Ω ($\times 2$)
- Capacitor: 100 μ F electrolytic (motor supply filtering)
- Breadboard and jumper wires
- Multimeter
- Oscilloscope with probes

Software

- STM32CubeIDE with PWM timer project

! PRE-LAB SECTION

Complete the following **BEFORE** arriving at the lab session.

Pre-Lab Checklist

Complete before arriving at the lab:

- ☐ Read theory slides on flyback diodes and H-bridge operation
- ☐ Review MOSFET datasheet: identify Gate, Drain, Source pins
- ☐ Understand motor equivalent circuit (winding resistance + inductance)
- ☐ Review PWM timer configuration from Week 1

Pre-Lab Questions

1. Why can't you connect a motor directly to an MCU GPIO pin? _____
2. What happens to the energy stored in a motor's inductance when you suddenly turn off current? _____
3. Why is "shoot-through" dangerous in an H-bridge? _____

Pre-Lab Calculations

Complete these calculations before lab:

1. Motor Characterization

Using your motor's specifications (or measure with multimeter):

Winding resistance: $R_m = \text{_____} \Omega$ (measure with motor stopped)

Stall current at 6V:

$$I_{stall} = \frac{6V}{R_m} = \text{_____} mA$$

2. MOSFET Power Dissipation

For IRLZ44N with $R_{ds(on)} = 50\text{m}\Omega$ at 3.3V gate drive:

Power at stall current:

$$P_{diss} = I_{stall}^2 \times R_{ds(on)} = \text{_____} mW$$

Is this acceptable without heatsink? YES / NO

3. Flyback Diode Selection

Diode must handle:

- Forward current $> I_{stall} = \text{_____} \text{ mA}$
- Reverse voltage $> V_{motor} = \text{_____} \text{ V}$

1N5819 ratings: If = 1A, $V_r = 40\text{V}$

Is 1N5819 suitable? YES / NO

4. PWM Frequency

For 10 kHz PWM on 84 MHz timer:

$$ARR = \frac{84,000,000}{10,000} - 1 = \text{_____}$$

Duty cycle resolution: $Steps = ARR + 1 = \text{_____}$ steps (0-100%)

i IN-LAB SECTION

The following sections are completed during the lab session.

Step 1.1: Measure Motor Parameters

Before building the circuit, characterize your motor:

1. Measure winding resistance with multimeter (motor stopped)
2. Connect motor directly to bench supply at low voltage (2V)
3. Gradually increase to 6V, note no-load current

Parameter	Measured Value
Winding resistance (R_m)	_____ Ω
No-load current at 6V	_____ mA
No-load speed (estimated)	_____ RPM

Step 1.2: Configure PWM Timer

Set up Timer 2, Channel 1 for 10 kHz PWM — you will use this signal to switch the motor:

STM32CubeIDE Configuration:

- Timer: TIM2
- Channel 1: PWM Generation
- Prescaler: 0
- Counter Period (ARR): 8399
- Pulse (CCR1): 4200 (50% duty)

Paste this into `main.c`:

```
/* USER CODE BEGIN 2 */
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1,
    (htim2.Init.Period + 1) * 50 / 100); // 50% duty
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1) {
    // Motor runs continuously - observe on scope
/* USER CODE END WHILE */
}
```

Step 1.3: Verify PWM Output

Before connecting to the motor, verify the PWM signal:

1. Connect scope CH1 to PA0
2. Confirm 10 kHz frequency and 50% duty cycle

Parameter	Expected	Measured
Frequency	10 kHz	_____ kHz
Duty cycle	50%	_____ %

Step 1.4: Build Low-Side Driver (WITHOUT Flyback Diode)

Build this circuit on your breadboard:

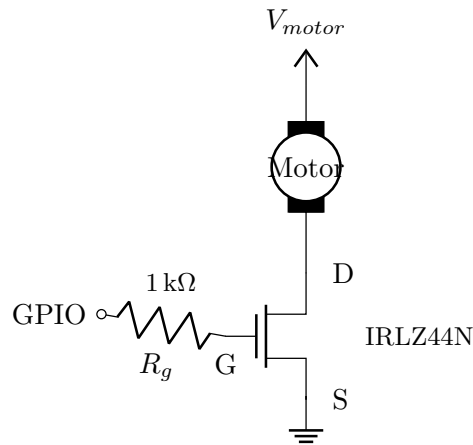


Figure 1: Low-Side MOSFET Motor Driver (No Flyback)

Build exactly as shown — **no flyback diode yet**. We want to observe the kickback problem first.

Step 1.5: Observe Kickback Under PWM

Why This Works

With PWM switching the motor on and off 10 000 times per second, every turn-off edge produces a kickback spike from the motor's inductance. The oscilloscope will show a continuous stream of spikes — much more dramatic than a single manual toggle.

Oscilloscope Setup

- **CH1:** Gate signal (PA0) — shows PWM edges
- **CH2:** Drain voltage (motor negative terminal)
- **Ground:** Circuit GND
- **Coupling:** DC on both channels
- **Scale:** 5 V/div on CH1, **20 V/div on CH2**
- **Trigger:** Falling edge on CH1
- **Timebase:** 20 μs /div

Power up and observe:

1. You should see a voltage spike on CH2 at every falling edge of the PWM
2. Zoom in on a single turn-off edge — record the spike amplitude and duration

Observation	Value
Drain voltage when MOSFET ON	_____ V
Peak kickback spike at turn-off	_____ V
Spike duration	_____ μs
MOSFET $V_{ds(\text{max})}$ rating	55 V

Will This Destroy the MOSFET?

With a small hobby motor at 6 V, the kickback spike is typically 20–40 V — within the IRLZ44N's 55 V rating. The MOSFET absorbs the spikes through avalanche breakdown, which is safe short-term. But this is **not acceptable in a real design**: a larger motor, higher voltage, or longer wiring inductance can easily exceed $V_{ds(\text{max})}$ and destroy the transistor. Always add flyback protection.

Step 1.6: Add Flyback Diode

Power down and add the protection diode:

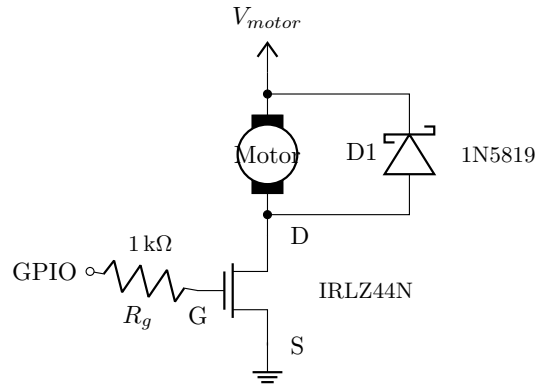


Figure 2: Low-Side Driver with Flyback Diode

Diode orientation: Cathode (band) toward +6V, Anode toward Drain. See diagram for placement.

Step 1.7: Verify Flyback Protection

Power up again with the same scope settings and compare:

Observation	Value
Peak voltage at turn-off	_____ V
Voltage clamp level	_____ V

Expected: Voltage should clamp to approximately $6\text{V} + 0.5\text{V} = 6.5\text{V}$

Compare with Step 1.5: Spike without diode was _____ V $\rightarrow$ now clamped to $\sim 6.5\text{V}$

Checkpoint

You should see clean turn-off edges with the drain voltage clamped near the supply rail. If spikes persist, check diode orientation (cathode band toward +6 V).

Part 2

PWM Speed Control

20 minutes

Step 2.1: Add Speed Stepping Code

Replace the Part 1 code. Each blue button press cycles: 0% → 25% → 50% → 75% → 100%.

```
/* USER CODE BEGIN PV */
static const uint8_t duty_pct[] = {0, 25, 50, 75, 100};
static uint8_t idx = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 2 */
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1) {
    if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET) {
        HAL_Delay(200);
        if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET) {
            idx = (idx + 1) % 5;
            __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1,
                (htim2.Init.Period + 1) * duty_pct[idx] / 100);
            while (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13)
                == GPIO_PIN_RESET);
        }
    }
}
/* USER CODE END WHILE */
}
```

Step 2.2: Motor Speed vs Duty Cycle

Duty Cycle	Motor Behavior	Relative Speed
0%	_____	0
25%	_____	_____
50%	_____	_____
75%	_____	_____
100%	_____	10 (max)

Step 3.1: CubeMX Configuration

Add two GPIO outputs for H-bridge direction control:

In CubeMX Pinout view:

- Set **PB4** → GPIO_Output (label: AIN1)
- Set **PB5** → GPIO_Output (label: AIN2)

Regenerate code, then keep TIM2 and USART2 from previous parts.

Note: PB4 is a JTAG pin (NJTRST) on STM32F103, but CubeMX automatically releases it when you assign it as GPIO_Output.

Step 3.2: Connect Motor Driver Module

Connect TB6612FNG (or L298N) module:

TB6612FNG Connections:

Module Pin	Connect To
VM	+6V bench supply
VCC	3.3V from Nucleo
GND	Common ground
STBY	3.3V (or GPIO for standby control)
PWMA	PA0 (PWM output)
AIN1	PB4 (direction control)
AIN2	PB5 (direction control)
AO1, AO2	Motor terminals

L298N alternative: ENA→PA0, IN1→PB4, IN2→PB5, VS→+6V, GND→common.

Step 3.3: Test Direction Control

Replace the Part 2 code with the following. Each button press cycles through: Forward 50% → Reverse 50% → Brake → Coast.

```

/* USER CODE BEGIN Includes */
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */

/* USER CODE BEGIN 0 */
static void motor_forward(uint8_t duty) {
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_RESET);
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1,
        (htim2.Init.Period + 1) * duty / 100);
}

static void motor_reverse(uint8_t duty) {
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_SET);
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1,
        (htim2.Init.Period + 1) * duty / 100);
}

static void motor_brake(void) {
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_RESET);
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 0);
}

static void uart_print(const char *msg) {
    HAL_UART_Transmit(&huart2, (uint8_t *)msg,
        strlen(msg), HAL_MAX_DELAY);
}

static const char *mode_names[] = {
    "Forward 50%", "Reverse 50%",
    "Forward 25%", "Forward 75%",
    "Reverse 25%", "Reverse 75%",
    "Brake",      "Coast"
};
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
MX_USART2_UART_Init();
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);

```

```

uint8_t mode = 0;
char buf[48];
uart_print("\r\n=== H-Bridge Control ===\r\n");
/* USER CODE END 2 */

/* USER CODE BEGIN WHILE */
while (1) {
    if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET) {
        HAL_Delay(200);
        if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET) {
            switch (mode) {
                case 0: motor_forward(50); break;
                case 1: motor_reverse(50); break;
                case 2: motor_forward(25); break;
                case 3: motor_forward(75); break;
                case 4: motor_reverse(25); break;
                case 5: motor_reverse(75); break;
                case 6: motor_brake(); break;
                case 7: motor_brake(); break; // coast
            }
            snprintf(buf, sizeof(buf),
                    "[%d/8] %s\r\n", mode+1, mode_names[mode]);
            uart_print(buf);
            mode = (mode + 1) % 8;
            while (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13)
                    == GPIO_PIN_RESET);
        }
    }
}
/* USER CODE END WHILE */
}

```

Open a serial terminal (115200 baud) to see mode changes.

Control logic for TB6612:

AIN1	AIN2	PWM	Motor Action
0	0	X	Brake
1	0	>0	Forward
0	1	>0	Reverse
1	1	X	Brake

Fill in results as you press the button:

Test	AIN1	AIN2	PWM	Result
Forward	1	0	50%	_____
Reverse	0	1	50%	_____
Brake	0	0	50%	_____
Coast (STBY=0)	X	X	X	_____

Observation: Does motor spin in opposite directions for Forward/Reverse?

YES / NO

Step 3.4: Speed Control in Both Directions

Continue pressing the button to cycle through 25% and 75% in both directions:

Direction	Duty Cycle	Motor Speed (1-10)
Forward	25%	_____
Forward	75%	_____
Reverse	25%	_____
Reverse	75%	_____

Step 3.5: Braking Test

Compare active braking vs coasting (last two button presses):

1. Run motor at 75% forward (press until “Forward 75%”)
2. Press → **Brake** — time to stop: _____ seconds
3. Run motor at 75% forward again
4. Press until → **Coast** — time to stop: _____ seconds

Observation: Which stops faster? BRAKE / COAST

Measurement Summary

ID	Measurement	Expected	Measured	Pass
M1	Motor winding resistance	5-15 Ω		
M2	Flyback spike (no diode)	20-40V		

ID	Measurement	Expected	Measured	Pass
M3	Clamped voltage (with diode)	~6.5V		
M4	PWM frequency	10 kHz		
M5	H-bridge forward works	Motor CW		
M6	H-bridge reverse works	Motor CCW		
M7	Brake vs coast stopping	Brake faster		

Failure Injection Test

Demonstrate understanding by predicting failure modes:

Scenario	What Happens?	How to Detect/Prevent?
Flyback diode removed		
Flyback diode reversed		
MOSFET gate direct to GPIO (no resistor)		
Motor connected directly to GPIO		
H-bridge shoot-through (both high ON)		

Analysis Questions

Answer these during the lab (2-3 sentences each):

1. Why does the flyback spike voltage exceed the supply voltage? _____
2. Why does motor speed not linearly follow PWM duty cycle at low duty cycles?

3. What is the advantage of active braking over coasting? _____

Common Issues and Solutions

Issue	Symptom	Solution
Motor doesn't run	No movement	Check MOSFET orientation, gate signal

Issue	Symptom	Solution
Motor runs but no control	Always full speed	PWM not connected, check timer config
MOSFET gets very hot	Overheating	Check gate drive voltage, add heatsink
Flyback diode doesn't clamp	High spike persists	Diode reversed, check orientation
H-bridge only one direction	Can't reverse	Check AIN1/AIN2 wiring
Motor jerky at low speed	Stuttering	Increase PWM frequency
Bench supply trips	Overcurrent	Motor stalled, reduce duty cycle

Cleanup Checklist

Before leaving:

- ☐ Turn off bench power supply
- ☐ Disconnect motor from all circuits
- ☐ Return oscilloscope probes
- ☐ Clean up breadboard and components

Key Takeaways

1. **Always use flyback protection** - inductive loads destroy switches without it
2. **MOSFETs need proper gate drive** - logic-level MOSFETs for 3.3V control
3. **PWM provides efficient speed control** - motor inductance smooths current
4. **H-bridge enables bidirectional control** - but beware of shoot-through
5. **Motor driver ICs simplify design** - built-in protection and dead-time